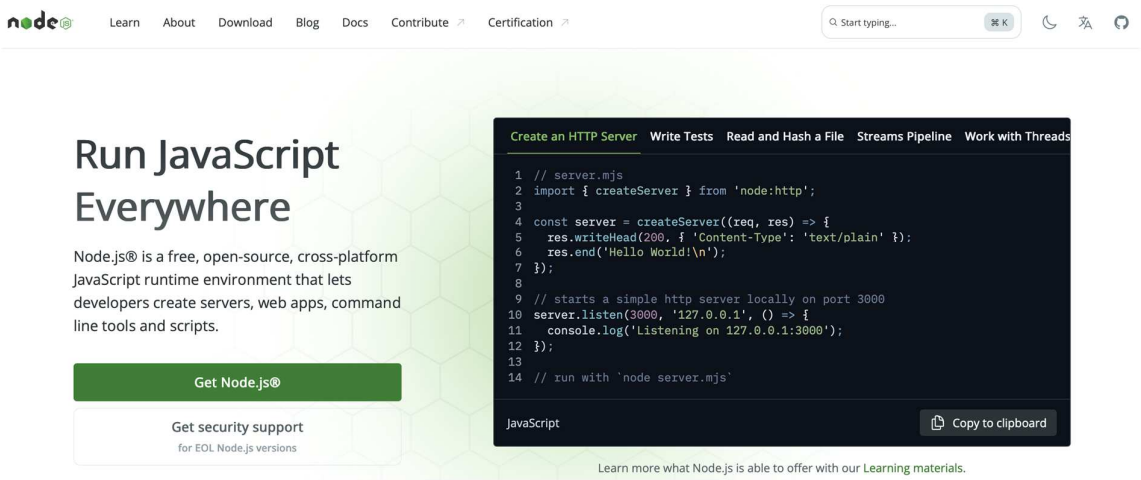
Bài 1

LẬP TRÌNH CLIENT/SERVER SỬ DỤNG SOCKETIO

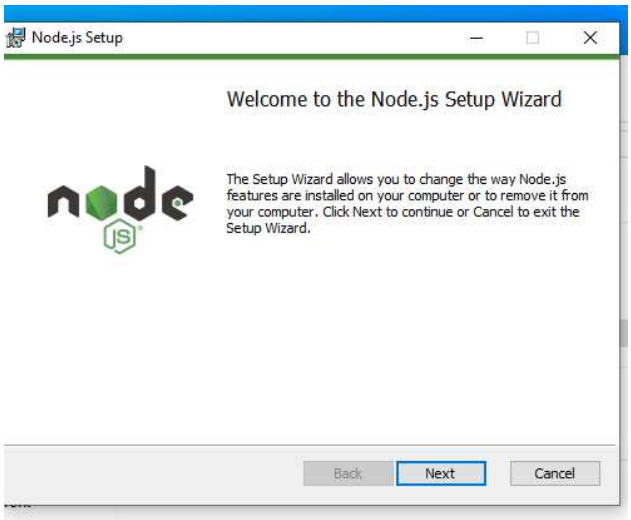
1. CÀI ĐẶT MÔI TRƯỜNG PHÁT TRIỂN

1.1 Cài đặt Javascript runtime (Node.js)

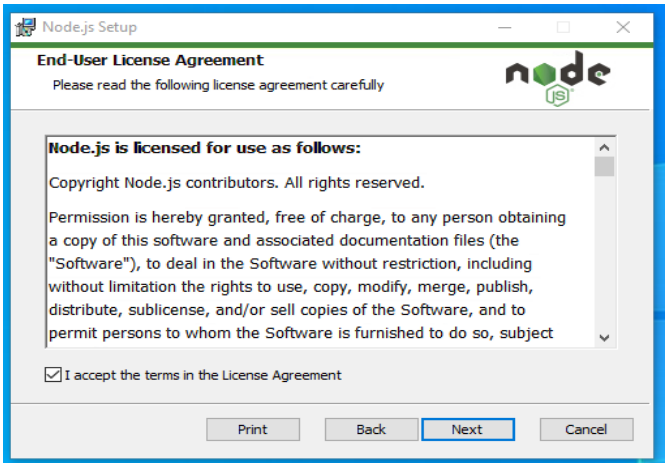
Tải Node.js từ địa chỉ https://nodejs.org/ có giao diện như hình sau:



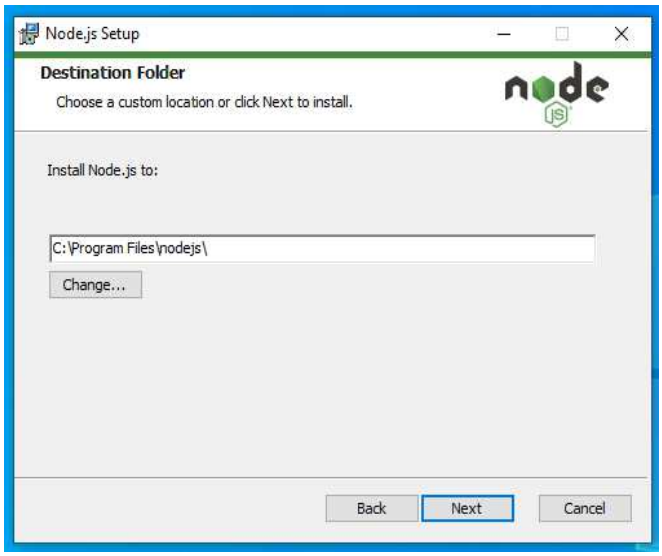
Mở file cài đặt Node.js vừa tải xuống và chọn Next để tiếp tục:



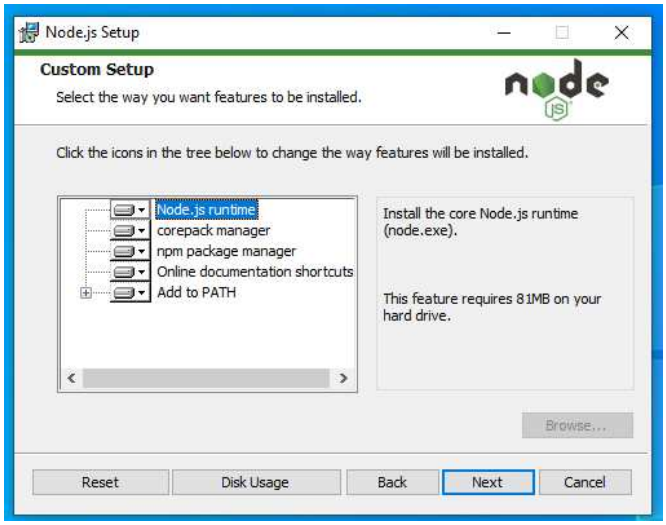
Chọn I accept the terms in the Licence Agreement, chọn Next để tiếp tục:



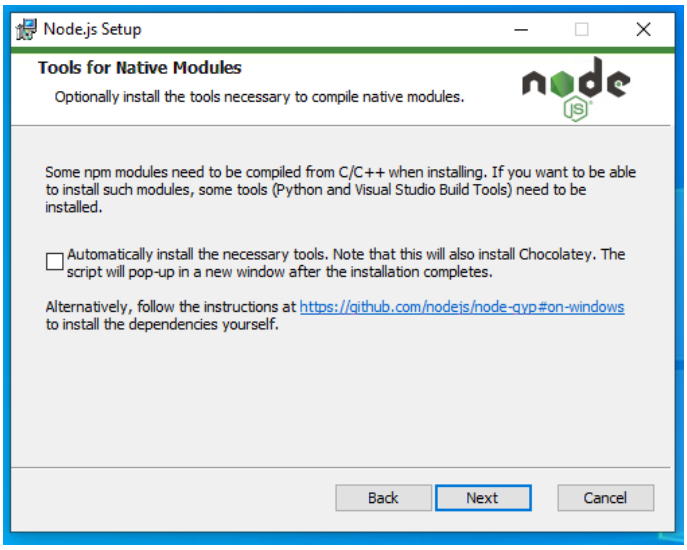
Chọn đường dẫn cài đặt và bấm Next để tiếp tục:



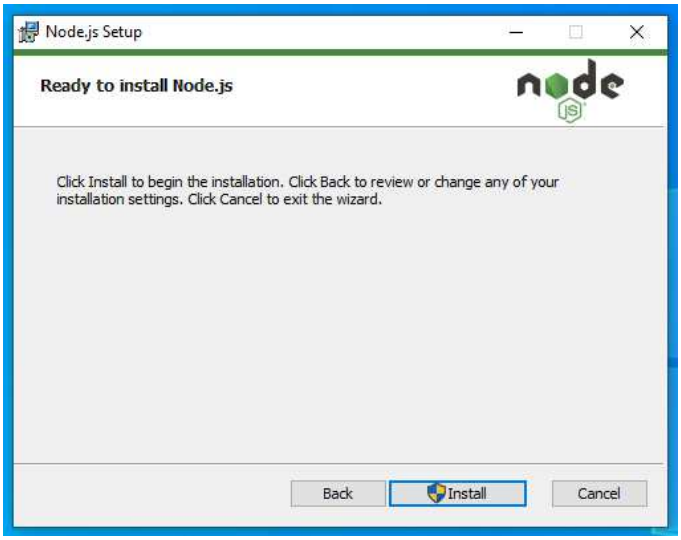
Tuỳ chọn các mô-đun cài đặt, bỏ qua bước này để cài mặc định, chọn Next để tiếp tục:



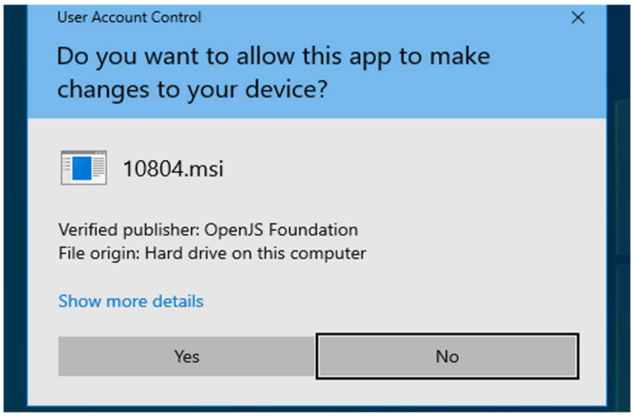
Chọn *Next* để bỏ qua bước kế tiếp.



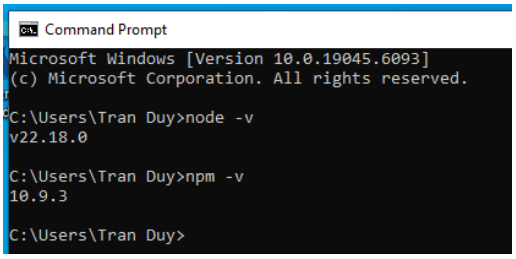
Chọn *Install* để bắt đầu quá trình cài đặt.



Chọn *Yes* để hoàn thành cài đặt.

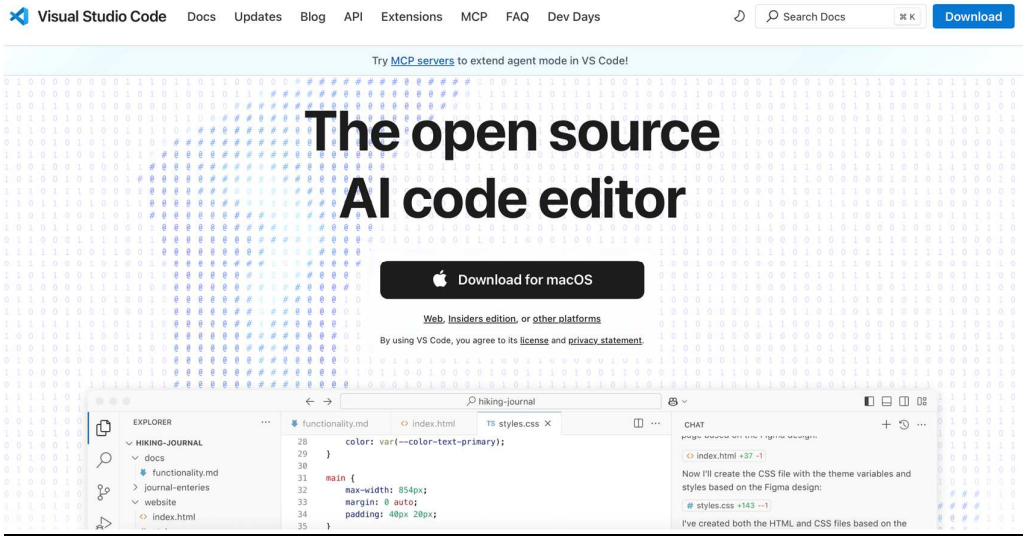


Sau khi cài đặt hoàn thành, từ cửa sổ Command Prompt gõ hai lệnh như hình sau để kiểm tra phiên bản ***Node*** và ***npm*** đã cài đặt, nếu trả về số phiên bản là quá trình cài đặt đã thành công.

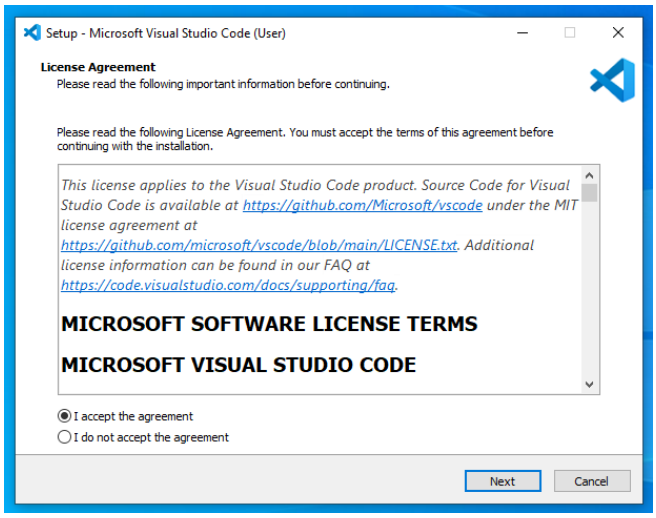


1.2. Cài đặt phần mềm Visual Studio Code

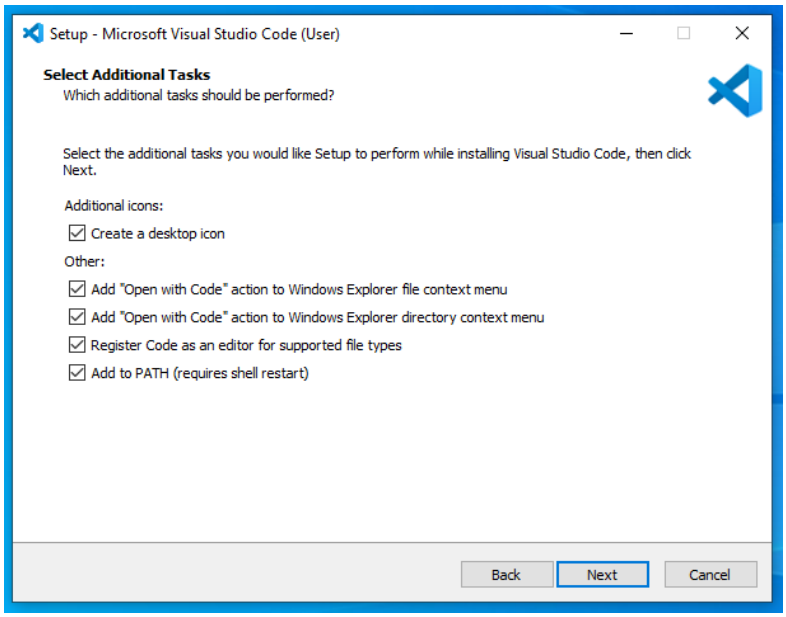
Tải Visual Studio Code (VSCode) từ địa chỉ https://code.visualstudio.com/, giao diện như hình sau:



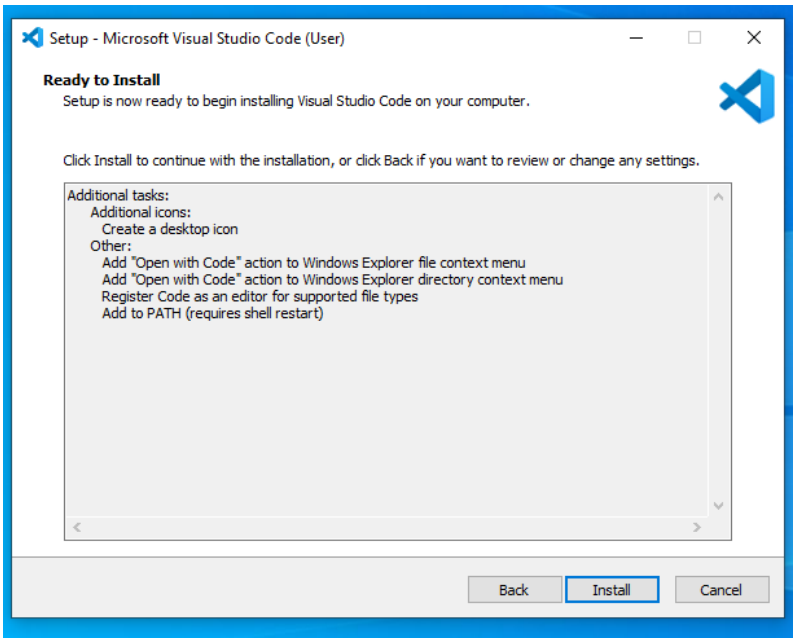
Mở tập tin vừa tải xuống, chọn ***I accept the agreement*** và chọn ***Next*** để tiếp tục.



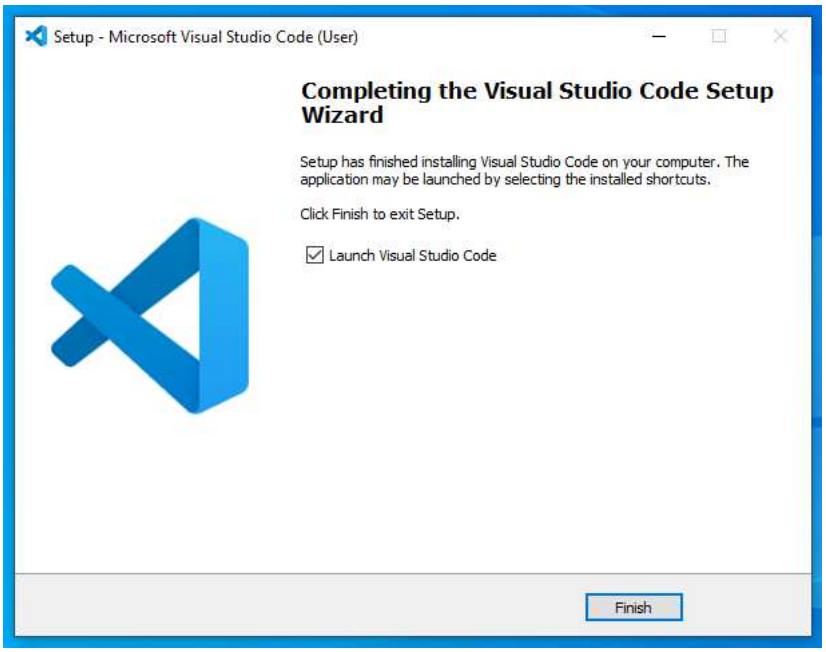
Chọn các tác vụ như hình, bỏ chọn ***Create a desktop icon*** nếu không muốn tạo biểu tượng phần mềm trên màn hình nền. Chọn ***Next*** để tiếp tục:



Chọn Install để bắt đầu cài đặt:



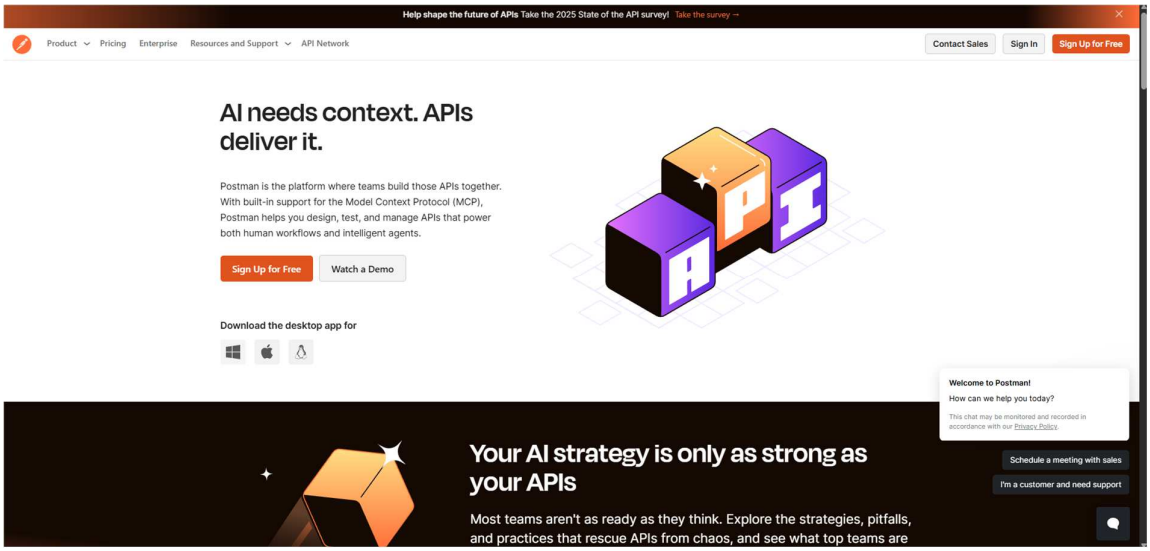
Chọn Finish để hoàn thành cài đặt.



Tìm hiểu cài đặt thêm các extension cần thiết như: Prettier, ESLint (tùy chọn).

1.3. Cài đặt Postman để tạo các yêu cầu HTTP kiểm tra hệ thống client/server

Tải phần mềm Postman từ địa chỉ https://www.postman.com/, tiến hành cài đặt và tìm hiểu cách tạo các yêu cầu HTTP.



2. LẬP TRÌNH CLIENT/SERVER DỰA TRÊN NỀN TẢNG SOCKETIO

2.1 Chuẩn bị

Các nhóm cần chuẩn bị:

- Một mạch ESP32 kèm cáp USB nạp chương trình, 4 LED, 2 biến trở 10k, breadboard và các dây cáp kết nối mạch phù hợp;
- Một máy tính đã cài đặt mô trường lập trình, phần mềm Arduino đã cài đặt sẵn:
  - Thư viện hỗ trợ mạch ESP32 trên Arduino phiên bản mới nhất;
  - Thư viện ArduinoJson;
  - Thư viện WebSockets phiên bản 2.7.0 do Markus Sattler phát triển;
  - Thư viện SocketIoClient phiên bản 0.3 do Vincient Wyszynski phát triển.

2.2 Lập trình máy khách ESP32 kết nối máy chủ JS để gửi dữ liệu

Các nhóm thực hiện lần lược các bước sau:

- Bước 1: Lập trình máy chủ JS sử dụng chương trình mẫu sau:

Chương trình mẫu 1.1: js_server_01.js

```
const io = require("socket.io")(3000, {
  origins: "*:*"
});
io.on("connection", (socket) => {
  console.log("ESP32 connected:", socket.id);

  socket.on("hello", (msg) => {
    console.log("ESP32 says:", msg);
  });

  socket.on("telemetry", (data) => {
    console.log("Telemetry:", data);
  });
});
```

Mở cửa sổ Terminal và thực hiện lệnh cài đặt thư viện SocketIO bằng lệnh:

```
npm install socket.io@2.4.1
```

Và thực hiện lệnh sau để khởi động máy chủ JS:

```
node js_server_01.js
```

- *Bước 2*: Lập trình máy khác ESP32 sử dụng chương trình mẫu sau:

Chương trình mẫu 1.2: esp32_client_01.ino

```
#include <Arduino.h>
#include <WiFi.h>
#include <SocketIoClient.h>

// Thông tin WiFi
const char* ssid = "Your SSID";
const char* password = "Your Password";

// Địa chỉ server Socket.IO
const char* host = "Your server IP";
const uint16_t port = 3000;

SocketIoClient socket;
void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nConnected!");

  socket.begin(host, port);
  socket.on("connect", [] (const char * payload, size_t length) {
    Serial.println("Connected to Socket.IO server!");
    socket.emit("hello", "\"Hello from ESP32\"");
  });
}
void loop() {
  socket.loop();
  // Gửi dữ liệu về máy chủ JS mỗi 5 giây
```

```
  static unsigned long lastSend = 0;
  if (millis() - lastSend > 5000) {
    lastSend = millis();
    String telemetry = "{\"temp\":25.5,\"hum\":60}";
    socket.emit("telemetry", telemetry.c_str());
    Serial.println("Sent telemetry");
  }
}
```

Nạp chương trình cho ESP32, ghi nhận lại hình ảnh kết quả trên cả màn hình Serial monitor và Terminal.

Câu hỏi:

Câu 1.1: Giải thích cơ chế truyền thông giữa máy chủ JS và máy khách ESP32 dựa trên kết quả vừa ghi nhận.

Câu 1.2: Giả sử máy khách cũng đang thu thập áp suất khí quyển và nồng độ CO2 trong không khí và gửi về máy chủ sau mỗi 10 giây. Người học hãy trình bày rõ nội dung hiệu chỉnh chương trình máy chủ JS và máy khách ESP32; thực hiện chương trình và ghi nhận lại (các) hình ảnh kết quả.

Câu 1.3: Hãy thay thế giá trị nhiệt độ và độ ẩm bằng giá trị giả lập trừ điện áp đọc được từ hai biến trở. Người học hãy trình bày rõ nội dung hiệu chỉnh chương trình của máy khách ESP32; thực hiện chương trình và ghi nhận lại (các) hình ảnh kết quả.

Bước 3: Bổ sung tính năng nhận lệnh điều khiển từ máy chủ cho máy khách ESP32. Người học hãy cập nhật lại chương trình mới cho máy chủ JS và máy khách ESP32 như sau:

- *Chương trình máy khách ESP32*:

Chương trình mẫu 1.3: esp32_client_02.ino

```
#include <Arduino.h>
#include <WiFi.h>
#include <SocketIoClient.h>

// Thông tin WiFi
const char* ssid = "Your SSID";
const char* password = "Your Password";
```

```
// Địa chỉ server Socket.IO
const char* host = "Your server IP";
const uint16_t port = 3000;

SocketIoClient socket;
const int LED_PIN = 2; // onboard LED pin
void handleControl(const char * payload, size_t length) {
  Serial.print("Received control: ");
  Serial.println(payload);
  String s(payload, length);
  if (s.indexOf("\"led\":true") >= 0) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("LED ON");
  } else if (s.indexOf("\"led\":false") >= 0) {
    digitalWrite(LED_PIN, LOW);
    Serial.println("LED OFF");
  }
}

void setup() {
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  Serial.begin(115200);
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nConnected!");
  socket.begin(host, port);
```

```
  socket.on("connect", [] (const char * payload, size_t length) {
    Serial.println("Connected to Socket.IO server!");
    socket.emit("hello", "\"Hello from ESP32\"");
  });
  socket.on("control", handleControl);
}

void loop() {
  socket.loop();
  // Gửi dữ liệu mô phỏng mỗi 5 giây
  static unsigned long lastSend = 0;
  if (millis() - lastSend > 5000) {
    lastSend = millis();
    float temp = random(200, 350) / 10.0; // 20.0 - 35.0
    float hum = random(400, 700) / 10.0;  // 40.0 - 70.0
    char buf[64];
    snprintf(buf, sizeof(buf), "{\"temp\":%.1f,\"hum\":%.1f}", temp, hum);
    socket.emit("telemetry", buf);
    Serial.print("Sent telemetry: ");
    Serial.println(buf);
  }
}
```

Nạp chương trình cho ESP32 để khởi động máy khách.

- *Chương trình máy chủ JS:*

Chương trình mẫu 1.3: js_server_02.js

```
const io = require("socket.io")(3000, {
  origins: "*:*"
});

// Keep a simple toggle state for demo control
let ledState = false;
```

```
io.on("connection", (socket) => {
  console.log("ESP32 connected:", socket.id);

  socket.on("hello", (msg) => {
    console.log("ESP32 says:", msg);
  });

  socket.on("telemetry", (data) => {
    console.log("Telemetry from", socket.id, ":", data);
    // try parse telemetry and act upon conditions
    try {
      const t = JSON.parse(data);
      if (t && typeof t.temp === 'number') {
        // example: if temperature too high, force LED on
        if (t.temp > 30) {
          socket.emit("control", JSON.stringify({ led: true }));
          console.log("Sent control -> led ON to", socket.id);
        }
      }
    } catch (e) {
      // ignore parse errors
    }
  });
});

// Broadcast a control command to all clients every 15s (toggle)
setInterval(() => {
  ledState = !ledState;
  io.emit("control", JSON.stringify({ led: ledState }));
  console.log("Broadcast control led =", ledState);
}, 15000);
```

Khởi động máy chủ, quan sát và trình bày lại kết quả. Người học lần lượt trả lời các câu hỏi sau:

Câu 1.4: Hãy giải thích cơ chế gửi lệnh điều khiển từ máy chủ JS đến máy khách ESP32.

Câu 1.5: Nâng cấp hệ thống để máy chủ có thể điều khiển chớp tắt 4 LED ở cùng và khác tốc độ. Trình bày lại những nội dung cần nâng cấp.

Câu 1.6: Lắp thêm một LED khác vào mạch, hiệu chỉnh chương trình máy chủ và máy khác để máy chủ có thể gửi lệnh điều khiển độ sáng của LED lắp thêm. Trình bày lại những nội dung đã hiệu chỉnh.

Câu 1.7: Lắp thêm một biến trở vào mạch, hiệu chỉnh chương trình để máy khách ESP32 đọc giá trị chuyển đổi ADC tại ngõ ra của biến trở gửi về máy chủ. Từ giá trị này, máy chủ JS sẽ tính ra giá trị điện áp và gửi về máy khách. Trình bày lại những nội dung đã hiệu chỉnh và một số hình ảnh kết quả.